

Rodrigo Fernandez

Senior .NET Backend Engineer

Katy, Texas | +1 (407) 801-1287 | <https://www.linkedin.com/in/rodrigo-bermejo-fern%C3%A1ndez-9a5a0664/>
rodrigobfdev@gmail.com

Summary

Senior .NET Backend Engineer with 13+ years of experience designing, modernizing, and supporting enterprise backend systems using **C#, ASP.NET Core, .NET Framework, SQL Server, PostgreSQL, Azure, AWS, Docker**, and message-driven architecture. Strong background in API development, legacy modernization, performance tuning, billing and provisioning workflows, secure multi-tenant systems, CI/CD, observability, and production troubleshooting. Experienced in delivering scalable backend services for SaaS, cloud operations, finance, healthcare, logistics, and e-commerce platforms.

Skills

- **Backend:** C#, ASP.NET Core, .NET Core, .NET Framework, Web API, REST APIs, gRPC, Minimal APIs, background workers, Windows Services
- **Databases:** SQL Server, PostgreSQL, Entity Framework Core, Dapper, T-SQL, stored procedures, indexing, query optimization, migrations
- **Cloud & DevOps:** Azure App Service, Azure Functions, Azure Service Bus, AWS EC2, S3, Lambda, CloudWatch, Docker, GitHub Actions, Azure DevOps, CI/CD
- **Architecture:** Clean Architecture, Domain-Driven Design, CQRS, repository pattern, unit of work, microservices, event-driven systems, API versioning
- **Security & Quality:** OAuth2, OIDC, JWT, RBAC, Serilog, Application Insights, OpenTelemetry, xUnit, NUnit, Moq, integration testing, SonarQube

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Designed and developed **ASP.NET Core** backend services for cloud hosting, billing, provisioning, identity, and customer operations workflows.
- Modernized legacy **.NET Core 3.1** services to **.NET 6/8 LTS**, resolving package conflicts, middleware issues, authentication regressions, and deployment risks.
- Improved API performance by optimizing **EF Core** queries, adding pagination, reducing over-fetching, tuning **SQL Server** indexes, and improving database access patterns.
- Built reliable background processing workflows using **Azure Service Bus**, Hangfire, retry policies, idempotency keys, and poison-message handling.
- Resolved production **DbContext** concurrency issues by correcting service lifetimes, improving transaction boundaries, and strengthening async disposal patterns.
- Implemented secure REST APIs using **JWT, OAuth2/OIDC** validation, role-based authorization, request signing, and centralized authorization handlers.
- Improved observability with **Application Insights**, CloudWatch, Serilog structured logs, metrics, and correlation IDs across APIs, workers, and databases.
- Refactored monolithic backend modules into cleaner service boundaries using repository, unit-of-work, mediator, and domain-service patterns.

- Strengthened CI/CD pipelines with **GitHub Actions**, Docker build caching, automated tests, migration checks, and rollback procedures.
- Reduced memory pressure in high-throughput file-processing APIs by streaming payloads, limiting upload sizes, and removing unnecessary large in-memory buffers.
- Delivered subscription, audit-log, customer-notification, and billing-related backend features with versioned APIs and database migration planning.
- Mentored engineers on C#, async programming, dependency injection, secure configuration, testing, and production troubleshooting.

NIX United — Software Engineer *(Oct 2016 – Mar 2020)*

- Developed enterprise backend APIs using **ASP.NET Core**, **.NET Framework**, SQL Server, RabbitMQ, and Azure DevOps for finance, logistics, and workflow platforms.
- Migrated selected Web API and WCF modules toward **.NET Core** by introducing .NET Standard shared libraries, dependency injection, and compatibility wrappers.
- Improved batch job reliability by redesigning retry logic, SQL timeout handling, exception classification, and alerting for reconciliation and synchronization workflows.
- Fixed recurring SQL deadlocks by reviewing execution plans, narrowing transaction scope, adding covering indexes, and improving repository methods.
- Built message-driven integrations using **RabbitMQ** exchanges, durable queues, dead-letter routing, and idempotent consumers.
- Created EF Core and SQL migration procedures with rollback scripts, seed-data validation, and release checklists.
- Strengthened API security with JWT authentication, refresh-token rotation, role-based access rules, password hashing improvements, and audit trails.
- Increased backend test coverage using xUnit, Moq, integration tests, and containerized test databases.
- Resolved serialization and timezone defects by standardizing UTC storage, explicit JSON contracts, contract tests, and backward-compatible DTO versioning.
- Supported Azure DevOps pipelines, NuGet package publishing, secure secrets, environment configuration, and production approval gates.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built backend modules for customer portals, CMS platforms, and e-commerce systems using **ASP.NET MVC**, Web API, .NET Framework, SQL Server, and IIS.
- Upgraded older **.NET Framework** applications, resolving NuGet conflicts, binding redirects, web.config issues, and deprecated library usage.
- Improved checkout and order-processing performance through SQL index tuning, catalog-data caching, and reduced database round trips.
- Fixed session expiration and cookie-authentication issues by reviewing IIS settings, machine keys, timeout configuration, and login-state handling.
- Implemented payment callback, invoice, email-notification, and admin-reporting features with validation and defensive error handling.
- Created stored procedures, views, and Dapper-based data access methods for reporting workflows where Entity Framework was difficult to optimize.
- Added structured logging, custom exception filters, and user-friendly error handling to improve production diagnostics.
- Coordinated with frontend developers on REST payloads, status codes, validation messages, and backward-compatible endpoint changes.

- Improved deployment stability by documenting IIS setup, app pool settings, connection strings, publishing steps, and rollback procedures.

Aurio — Junior Software Developer (*Oct 2012 – Dec 2013*)

- Maintained internal admin systems using C#, ASP.NET MVC, Web Forms, ADO.NET, SQL Server, and IIS.
- Implemented CRUD screens, reporting filters, export functions, and permission checks for internal business users.
- Fixed null-reference, validation, and database-connection issues in legacy data access code.
- Improved report performance by simplifying stored procedures, adding indexes, and reducing repeated queries.
- Assisted with migration from Web Forms to ASP.NET MVC by separating presentation logic, validation rules, and reusable services.
- Wrote SQL scripts, stored procedures, and maintenance jobs to clean duplicate records and support reporting needs.
- Supported IIS deployments by updating web.config values, verifying connection strings, reviewing Windows event logs, and checking app pool behavior.

Microsoft — Software Developer Intern (*Jun 2012 – Sep 2012*)

- Contributed to internal engineering utilities using C#, ASP.NET MVC, JavaScript, and SQL Server.
- Assisted with bug reproduction, log review, and small backend fixes for internal dashboard tools.
- Implemented UI updates and backend validation improvements under senior engineer code review.
- Prepared documentation for setup steps, known issues, and troubleshooting paths to support future onboarding.

Education

Texas State University

Bachelor's Degree in Computer Science (*Sep 2008 – May 2012*)